How to use geolens

Running a project area from ArcGIS Pro or a shell, keeping a long run going, and reading what it makes

Version 0.45 (beta), October 2026

For	Anyone running geolens on a project area (a field office, a ranch, a district) or on one DEM, from ArcGIS Pro, a command window or a notebook.
What you need	Windows 10 or 11 with 32 GB of memory or more (64 GB for a whole field office at six HUC12s at a time); a data drive with room (Section 1.3); internet access to the USGS, USDA, NOAA, NIFC and PRISM services; ArcGIS Pro 3.x if you want the toolbox (not needed for anything else).
Companions	`START_HERE.md` (ten-minute tour), `RUNNING.md` (every parameter), *geolens methods* (`docs/geolens_methods_v0.45.pdf`: how each result is made and how far to trust it).

1 Before the first run

1.1 Install

1. Unzip the release somewhere permanent on the data drive, for example `S:\geolens\gl45`. Each release goes in a folder of its own: a run in progress uses the files of the release it was started from, so never unzip a new release over a folder a run is using.
2. Double-click **`install_windows.bat`** once. The computer needs no Python and no admin rights: with conda on it (Miniforge, Anaconda or Miniconda) the installer builds a conda environment named `geolens`; without conda it builds geolens's own Python in `env` beside the release. It takes 5–15 minutes, mostly downloading, and tests itself on a synthetic valley. Updating later: unzip the new release into a new folder and run its installer; the environment is kept.
3. **Or portable, on an external drive:** double-click **`setup_portable.bat`** instead (or run `install_windows.bat --portable`). Everything, Python included, stays on the drive and nothing is installed on the computer, so the drive can be handed to a colleague. Keep the folder near the top of the drive (for example `E:\geolens`): Windows limits a path to 260 characters.

1.2 Add the toolbox to ArcGIS Pro

1. In the Catalog pane, right-click *Toolboxes* ▸ *Add Toolbox* and choose `arcgis_toolbox\geolens.pyt` in the release folder.
2. Run *0 Setup* ▸ *Find the geolens environment*. It should report "geolens 0.45.2 found". If it cannot find the environment, give it the `python.exe` the installer printed; it remembers it.
3. Run *0 Setup* ▸ *Check internet services* once on a new computer or network: it says which of the download services this machine can reach.

4. Only for the Sentinel-1 radar tool: *0 Setup* ▸ *Save Earthdata login* with your own free NASA Earthdata account. The password goes into your Windows profile, never into geolens or its records.

The tools run geolens in its own environment, so nothing is installed into ArcGIS Pro's Python and a run too big for memory cannot take ArcGIS Pro down.

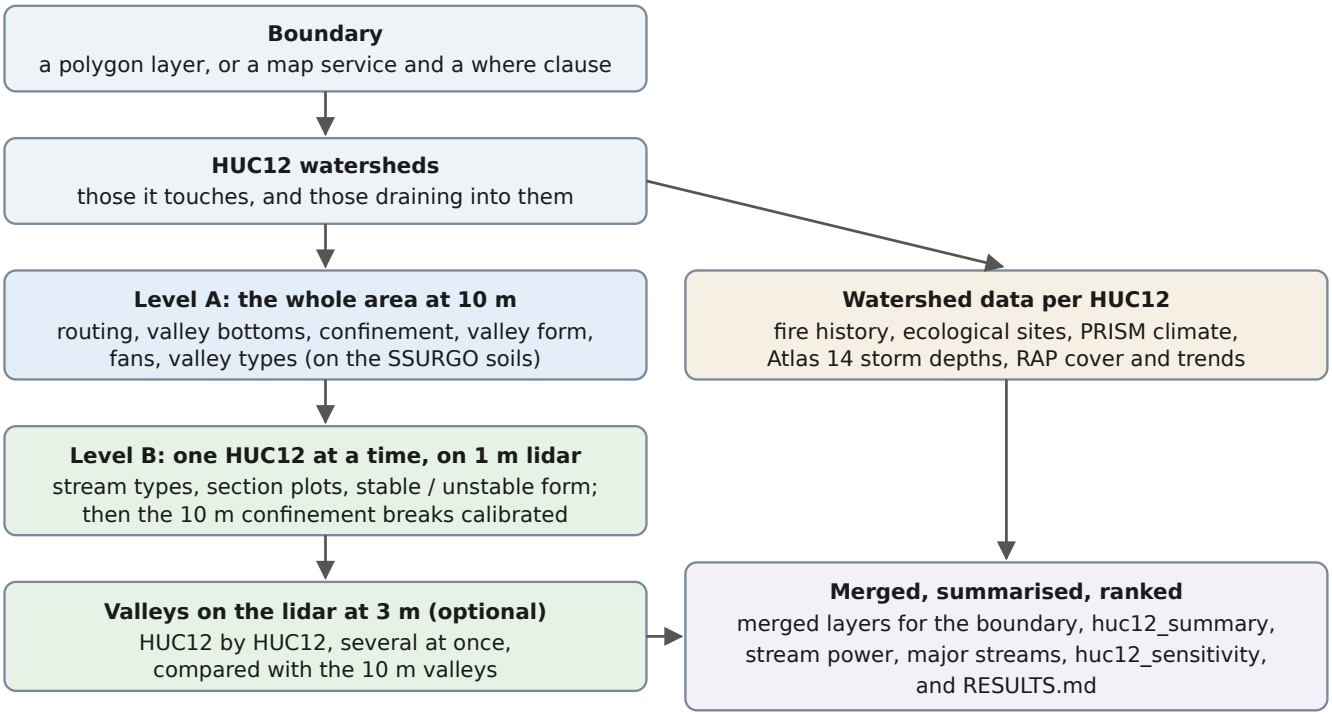
1.3 Time and space to plan for

Figures from a 41,000 km² BLM field office (309 HUC12s inside the boundary, 446 with those draining into it) on a 128 GB workstation.

Part	Time	Disk
Level A: the whole area at 10 m (routing, valleys, valley types)	about 2 hours	about 10 GB
SSURGO soils and the watershed data package	20–40 minutes	small
Level B: stream types on 1 m lidar, one HUC12 at a time	3–5 minutes a HUC12; about 20 hours for 270	1 m mosaics are deleted after use unless kept: about 300 MB a HUC12 (80 GB for 270)
Valleys on the lidar at 3 m	a few minutes a HUC12, six at a time: hours for a field office	a few GB, plus the 1 m mosaics if kept
Merging, the HUC12 summary, stream power and the sensitivity table	15–30 minutes	a few GB

Keep at least 50 GB free on the project's drive; the lidar valley pass stops cleanly, and can be carried on, if the free space falls under 30 GB. Temporary files go to the computer's TEMP folder unless the launcher sets them elsewhere (the project launcher keeps them on the data drive).

2 Running a project from ArcGIS Pro



How a project run goes. Each box is a step recorded in `project.json`, so a run that stops carries on from the last one finished.

2.1 The project tool

Open *1 Whole routine* ▸ *Run a project area (by HUC12)*.

Parameter	What to give it
Boundary	A polygon layer: a ranch, an allotment, a district.
or a map service layer URL, and a where clause	Instead of a layer: for example BLM's administrative units layer, `https://gis.blm.gov/arcgis/rest/services/admin_boundaries/BLM_Natl_AdminUnit/MapServer/3`, with a where clause such as `ADM_UNIT_CD='XXX00000'` for one office.
Project folder	A new, empty folder on the data drive. Everything the run makes goes here.
Start from an earlier project folder	Optional. A finished project's folder: its boundary, HUC12s, level A, level B, soils and watershed data are copied into the new folder, which carries on from them. The earlier folder is only read, never changed. Use it to rerun later steps with a new version without repeating a day of work.
Level B (stream types on lidar)	*Pilot: HUC12s that together represent the area* for a first look at a landscape project; *Every HUC12 with lidar* for the full product; *the most unconfined valley* for a stream project; *listed* for chosen HUC12s; *None* for valleys only.
Pilot HUC12s	How many pilots (3 by default).
Valley material	*Download USDA SSURGO soils* (the default and the best choice in the Southwest); USGS geology tends to call valleys bedrock.
Regional bankfull curve	*By location* picks Moody et al. (2003) for the arid Southwest, the generic US curve elsewhere.
Watershed data per HUC12	*All* (fire history, ecological sites, PRISM climate, Atlas 14 storm depths, RAP vegetation cover and trends), one of them, or none.
Run in its own window	Tick it for anything that takes hours (Section 2.2).

Under *Options*:

Option	Default	When to change it
Stream threshold, km²	1	The drainage area a stream starts at.
Stop level B after this many hours	0 (no limit)	To fit a run into a night; run again to carry on.
Section plots	on	Off only to save time: the plots are how a stream type is checked.
Fit the 10 m confinement breaks to the lidar	on	Leave on: level B's lidar sections calibrate the 10 m confinement classes.
Keep each HUC12's 1 m DEM	off	On when you will run the valleys on the lidar later, or want the DEMs: about 300 MB a HUC12, and the lidar pass then needs no download.
Major streams: drainage at the mouth, km²	25	The size of stream that gets its own line in `major_streams`.
Sensitivity weights file	built-in	A JSON file of summary column → weight, to re-rank the HUC12s.
Level A cell size, m	10	20 or 30 for a quick reconnaissance run on a laptop.
Valleys again on the lidar	None	*Pilot* (three headwater HUC12s, compared with 10 m) or *Every HUC12 with lidar*.
Lidar valleys: cell size, m	3	1–10 m. 3 m is the tested setting.
Lidar valleys: HUC12s at once	0 (automatic)	Fewer, to leave the computer usable; the automatic setting uses free memory and cores, at most six.
Level B again where the 1 m lidar mosaic had holes	off	On for a project run before v0.45 (see Section 6).

2.2 Long runs: in a window of their own

With *Run in its own window* ticked, the tool writes the run's settings into the project folder (`geolens_run.json`) with a launcher beside them (`run_geolens.bat`), starts the launcher in a black console window and finishes within seconds. Then:

- **Leave the console window open.** Closing it stops the run. Everything finished so far is kept.
- **ArcGIS Pro is free**, and can be closed: the run does not depend on it.
- The window shows progress bars and notes as it goes. If geolens stops with an error, the launcher waits a minute and carries on, up to five times.
- **After a reboot, a crash or a power cut, double-click `run_geolens.bat`** in the project folder. The run carries on from the last finished step or HUC12.
- One run at a time per project folder: a second one started on the same folder stops at once with "Another geolens run is working in this folder".
- For a run of a day or more: pause Windows Update for a week, keep the computer plugged in, and switch off sleep if your organisation allows (geolens also asks Windows not to sleep

while it works).

2.3 How far has it got?

Run *1 Whole routine* ▸ *Check on a project run* on the project folder at any time. It only reads the folder and never disturbs the run. It says:

- whether a run is working in the folder now, and since when;
- which steps are done;
- level B's HUC12s by status (done, failed, no lidar);
- for the valleys on the lidar: done, failed and still to go, the pace in HUC12s an hour and roughly how many hours are left;
- the free space on the drive, and the last warning.

From a command window the same is `geolens status <project folder>`.

2.4 The results on a map

When the run has finished, run *1 Whole routine* ▸ *Add a project's results to the map* with the project folder. It adds, to the active map, named and coloured:

Layer	On at first	What it is
HUC12 summary (all the numbers)	off	One row per HUC12: valley and soil makeup, slope, stream form, the watershed data.
HUC12 sensitivity	off	The HUC12s ranked high / moderate / low by the weighted components.
HUC12 status	off	Which HUC12s level B ran on, failed or had no lidar.
Valley extent (by valley type)	on	Each reach's valley floor as a polygon, coloured like its valley type.
Valley extent on the lidar at 3 m	on	The same from the lidar pass, where it ran.
Valley types	on	Every 10 m reach, coloured by Rosgen's 2022 fluvial landscape type.
Valley types on the lidar at 3 m	on	The lidar pass's reaches.
Stream types	off	Level B's reaches with a Rosgen stream type.
Stable / unstable form	off	Each level B reach's stream type judged against its valley type.
Major streams	on	One line per stream draining 25 km² or more at the mouth.
Validation sample	off	Reaches set aside to be scored by eye (Section 4.3), where a review made one.

Each name ends with the folder's name (or the text you give), so two runs can sit side by side. Run the tool again to re-colour: layers already on the map are not added twice and keep their on/off. Save the ArcGIS project to keep them.

The layers from the other tools are added and coloured when the tool finishes. Section and reach plots open in the pop-up: click a feature with the *Explore* tool.

3 Running without ArcGIS Pro

3.1 Drag and drop

Drag a boundary file (shapefile, GeoPackage, GeoJSON or KML) onto **`run_project.bat`** in the release folder, or double-click it and paste a map service URL and a where clause. The project folder is made beside the boundary. On a computer where geolens has no Python yet, it builds geolens's own first (once, 5–15 minutes) and then starts the project; a run that stops with an error is started again a minute later, up to five times, and carries on where it stopped.

3.2 A command window

Open the *Anaconda Prompt*, activate the environment (`conda activate geolens`) and:

```
geolens project boundary.gpkg --out S:\project\run1 --level-b all
geolens project <map service layer URL> --where "ADM_UNIT_CD='XXX00000'" --out S:\project\run1
geolens project <...> --out S:\project\run2 --copy-from S:\project\run1 --fine-valleys all --fine-
cell 3 --keep-lidar-dem
geolens status S:\project\run2
python -m geolens.job S:\project\run2\geolens_run.json
```

The last line runs the settings a launcher wrote, which is what `run_geolens.bat` does. Every option of the toolbox has a command-line flag (`geolens project --help`).

3.3 The notebook

`notebooks/geolens_start.ipynb` runs one DEM step by step with the numbers printed as it goes, and calibrates the confinement breaks interactively. Start it with `start_notebook_portable.bat` on a portable drive, or the command the installer printed.

4 Reading what it made

4.1 The project folder

Item	What it holds
`RESULTS.md`	Read this first: what ran at what resolution, the calibration, the lidar comparison, the watershed data steps, every warning.
`project.json`	The record of every step and HUC12: what makes a run resumable. Do not edit it while a run is working.
`merged\`	The layers for the boundary: `valley_types`, `stream_types`, `stream_type_sections`, `stream_stability`, `huc12_status`, `huc12_summary` (and `.csv`), `huc12_sensitivity` (and `.csv`), `major_streams`, `sensitivity_weights.json`, and with the lidar pass `valley_types_3m` and `valley_extent_3m`.
`A_landscape\`	Level A's full run, including `run\valley_extent_dir\valley_extent.gpkg` (the valley polygons).
`B_huc12\<code>\`	Level B per HUC12: stream types, sections, plots, stability.
`A_fine\<code>\` and `A_fine\comparison_3m.csv`	The lidar valley pass per HUC12, and its comparison with 10 m.
`data\`, `soils\`	The watershed data per HUC12 and the SSURGO soils.
`geolens_run.json`, `run_geolens.bat`, `geolens_run_result.json`	Written by *Run in its own window*: the settings, the launcher, and the result when it finished.

4.2 The fields that matter

Layer	Field	Meaning
valley types	`valley_type`	Rosgen's 2022 fluvial landscape type, e.g. `U-AL-AD` (unconfined, alluvial, terraced valley), `U-AL-IF` (inactive fan), `C-CO-VS` (confined, colluvial, V-shaped).
	`valley_type_1996`	The 1996 type, I–XI.
	`confidence`, `basis`, `alternatives`	How sure, why, and what else it could be (`_1996` for the 1996 type).
	`confinement`, `confinement_ratio`, `confinement_basis`	Confined / partly confined / unconfined, from the valley floor over the channel width, and what the floor width came from.
	`dem_cell_m`, `valley_resolved`	The cell the valley was measured on; false where the valley is under three cells wide.
	`stream_id`, `stream_power_w_m`, `unit_stream_power_w_m2`	The stream the reach is on; its stream power from the regional curve.
stream types	`stream_type`, `stream_type_full`, `stream_basis`, `stream_confidence`	Rosgen A–G with slope suffix (and substrate); whether the banks were measured, partly measured or regional; how sure.
stability	`form_status`, `form_confidence`, `form_note`	`stable`, `unstable`, `check` (not a type Rosgen lists for the valley: look at the plots) or `unknown`, and why.
`project.json`	`lidar_data`	Per HUC12, the share of it with 1 m lidar data (v0.45).

4.3 How far to trust it

Result	Use it as
Valley bottoms, valley extent, valley types	Mapped results, with the confidence each carries; 10 m resolves valleys three or more cells wide.
Confinement	Calibrated to the project's own lidar where level B ran.
Stream types and stable / unstable form	Screening: bankfull leans on the regional curve. Check the section plots before relying on a reach.
Sensitivity table	A ranking with proposed weights, to be argued with; change the weights file and rerun.
Valleys on the lidar at 3 m	The finer answer where the lidar covers; compare with 10 m in `comparison_3m.csv`.

When a review has made a **validation sample** (`review\validation_sample.gpkg` and `HOW_TO_SCORE.md`), score it in ArcGIS Pro: for each reach mark whether the valley type, the stream type and the form call look right, with your own stream type where it differs. Those scores are what tunes the classifiers next.

5 When something goes wrong

What you see	What to do
`install_windows.bat` says “No Python found”, or stops at once	That was the installer before v0.45.2. Use v0.45.2 or later, which builds geolens’s own Python when the computer has no conda, or double-click `setup_portable.bat`.
`setup_portable.bat` stops with “Package cache error”, or says the folder’s path is too long	Windows limits a path to 260 characters. Use v0.45.2 or later, and keep the geolens folder near the top of the drive (for example `E:\geolens`).
`setup_portable.bat` cannot download micromamba or the packages	Check the internet connection and run it again: it carries on. On a network that needs a proxy, set `HTTPS_PROXY` in a command window and run it from there.
“The geolens Python environment was not found”	Run *Find the geolens environment* and give it the environment’s `python.exe`; or rerun the installer.
“Another geolens run (process N) is working in …”	A run is already working in that folder (perhaps in a window behind others). Let it finish or close its window. A lock left by a run that died is taken over automatically.
The window says “Trying again in 60 seconds”	A step failed (often a service that did not answer). It carries on by itself; after five tries it stops and says so: double-click `run_geolens.bat` to carry on later.
“under 30 GB free” in RESULTS.md or the status	Free some space on the drive and run again; the run carries on.
A HUC12 “failed” in the status	The reason is in `project.json` and RESULTS.md. Failed HUC12s are retried once at the end and again on the next run.
Out of memory	The lidar pass retries a HUC12 at twice the cell by itself. For level A, use *Check memory* and a coarser *Level A cell size*.
“numba … is too new for pysheds”	In the geolens environment: `pip install "numba>=0.57,<0.62"`.
Layers come in grey or unnamed	Run *Add a project’s results to the map* on the folder, which names and colours them.
A number looks wrong	Send `RESULTS.md`, `project.json` and the run’s `runs\` folders (small text files): they hold the settings and warnings that explain most surprises.

6 Projects run before v0.45

Before v0.45 the 1 m lidar download could leave holes where lidar projects overlap or meet at a UTM zone line (*geolens methods*, Section 8.6), which left level B’s sections on empty ground in places (“unknown” reaches, and HUC12s that failed with “No channel sections could be sampled”). To bring such a project up to date without redoing the 10 m work, start a new folder from it and ask for both levels to be redone where it matters:

1. *Run a project area*, a new project folder, *Start from an earlier project folder* = the old one, the same boundary and settings.
2. Under *Options*: *Level B again where the 1 m lidar mosaic had holes* on; *Valleys again on the lidar* as you want them; *Keep each HUC12's 1 m DEM* on.
3. *Run in its own window* on.

Level B is then measured again on whole mosaics where the old ones skipped a lidar tile (most of a day for a field office), the confinement breaks are calibrated again, and the lidar valley pass reuses level B's mosaics. HUC12s whose old mosaics were whole are left as they were.